

Reto 2 - Christian Sanabria Jiménez – Junio 2026

Estrategia de Pruebas Automatizadas E2E para el sitio Demoblaze

Tabla de contenido

Reto 2 - Christian Sanabria Jiménez – Junio 2026.....	1
1. Descripción general de la estrategia	1
2. Objetivo del reto	2
3. Objetivo de la estrategia diseñada	2
4. Prerrequisitos y supuestos.....	2
5. Funcionalidades dentro del alcance.....	3
6. Tipos de pruebas y alcance	3
a. Datos de prueba	4
b. Métricas y criterios de salida.....	4
7. Funcionalidades fuera del alcance	4
8. Ambientes	5
9. Criterios de suspensión (aceptación, devolución, suspensión)	5
10. Tipos de pruebas y alcance	5
11. Datos de prueba	6
12. Métricas y criterios de salida.....	6
13. Matriz de riesgos	6
b. Riesgos positivos (Oportunidades).....	10
14. Escenarios de prueba BDD (Gherkin).....	11

1. Descripción general de la estrategia

Este documento contiene el enfoque global de aseguramiento de calidad mediante pruebas automatizadas para la aplicación ubicada en <https://www.demoblaze.com/index.html>. Se probarán diversas funcionalidades de la aplicación utilizando las herramientas WebDriverIO y Playwright con el objetivo de validar el funcionamiento correcto de las funcionalidades **y hacer pruebas de lo que la aplicación no debe permitir**.

DemoBlaze es una aplicación web de e-commerce: con un catálogo de productos por categorías (Phones, Laptops, Monitors), detalle de cada producto, un carrito de compras, un flujo de compra y funcionalidades de Login, Registro, Contacto y "About us".

2. Objetivo del reto

El ejercicio evalúa la capacidad del estudiante para:

- Diseñar escenarios BDD (Gherkin)
- Implementar automatización con dos herramientas distintas
- Integrar la ejecución en Jenkins con stages
- Generar evidencias automáticas

3. Objetivo de la estrategia diseñada

Asegurar que los flujos críticos (autenticación, navegación por categorías, carrito y compra) funcionen de forma consistente en los navegadores más usados, con datos de prueba controlados y ejecución repetible en ambiente local y CI (stages de Jenkins), junto con la generación de evidencias para casos de éxito y fallo.

4. Prerrequisitos y supuestos

• Entorno y dependencias

- Node.js LTS instalado (≥ 18).
- Para **WebdriverIO con Selenium**: JDK instalado (lo requiere el servicio @wdio/selenium-standalone-service).

- **Navegadores objetivo**: Chrome, Firefox en sus versiones recientes

Con el fin de ejecutar la estrategia se requiere:

1. Disponibilidad del sitio <https://www.demoblaze.com/index.html>

2. Acceso a herramientas de prueba vía npm: Webdriver
3. Scripts de pruebas desarrollados

- **Datos de prueba:**

- Usuarios nuevos generados dinámicamente para "Sign up".
- Productos identificados por nombre visibles en el catálogo.

- **Estado del sitio:** contenido y estructura de DemoBlaze disponibles y estables.

5. Funcionalidades dentro del alcance

Para el reto se desarrollaron escenarios de prueba para las siguientes funcionalidades:

1. **Registro (Sign up):** creación de usuarios nuevos y validación de alertas/mensajes. [\[demo blaze.com\]](https://demo blaze.com)
2. **Inicio de sesión (Log in):** acceso correcto y manejo de credenciales inválidas. [\[demo blaze.com\]](https://demo blaze.com)
3. **Catálogo por categorías:** filtrado por *Phones*, *Laptops* y *Monitors*. [\[demo blaze.com\]](https://demo blaze.com)
4. **Detalle de producto y "Add to cart":** agregado al carrito desde la ficha. [\[demo blaze.com\]](https://demo blaze.com)
5. **Carrito (Cart):** ver, eliminar ítems y calcular totales. [\[demo blaze.com\]](https://demo blaze.com)
6. **Checkout ("Place Order"):** completar datos, confirmar compra y validar confirmación. [\[demo blaze.com\]](https://demo blaze.com)
7. **Contacto (Contact):** envío desde el modal y validación de retroalimentación. [\[demo blaze.com\]](https://demo blaze.com)
8. **About us:** apertura del modal y verificación de contenido (incluye video).

6. Tipos de pruebas y alcance

- **Smoke:** entrada al sitio, abrir modales, navegar categorías, abrir un producto.
- **Funcionales/Flujo E2E:** sign up → login → agregar al carrito → checkout.
- **Negativas:** credenciales inválidas; checkout sin datos; agregar producto duplicado.
- **Regresión:** suite estable de los 10 escenarios (abajo), ejecutable en cada cambio.
- **Compatibilidad:** Chrome/Chromium y Firefox con WebdriverIO; Chromium/Firefox/WebKit con Playwright. [\[webdriver.io\]](https://webdriver.io), [\[playwright.dev\]](https://playwright.dev/)
- **Accesibilidad (exploratoria):** foco, tabulación y etiquetas en modales.

a. Datos de prueba

- Usuario aleatorio por build para **Sign up** (para evitar conflictos o nombres duplicados).
- Productos: se usa nombres visibles en la página (p. ej., "Sony vaio i5", "Samsung galaxy s6")

b. Métricas y criterios de salida

- **Tasa de éxito** de las pruebas $\geq 95\%$ en los navegadores establecidos.
- **Defectos críticos** (checkout/login) = 0 para aprobar
- Login exitoso con credenciales:
 - demoQA / demo123
- Login erróneo
 - usuario1 / clave1
- Navegación por categorías
 - Selección de una categoría y visualización de productos
 - Visualización del detalle de productos
- Compra de productos
 - Agregar producto al carrito
 - Ver carrito
 - Eliminar producto del carrito
 - Realizar la compra
 - Carrito vacío luego de completar la compra
- Logout exitoso

7. Funcionalidades fuera del alcance

En la presente estrategia se deja fuera del alcance:

1. **Pruebas de API** (si no hay endpoints públicos documentados).
2. **Performance avanzada** (carga, estrés, escalabilidad).
3. **Internacionalización** (cambios de idioma/moneda).
4. **Integraciones de pago reales** (tarjetas/PSP externas).

- Las validaciones relacionadas con la tarjeta y sus datos
- El registro de nuevos usuarios
-

8. Ambientes

Para las pruebas del reto se utilizará únicamente el ambiente público con el que se cuenta para la aplicación. No habrá ambientes adicionales de desarrollo o QA, el ambiente base contiene es el siguiente:

- **Local** (desarrollo del framework y smoke rápido).
- **CI** (ejecución paralela en Chrome/Firefox + reporte).
- **Base URL:** <https://www.demoblaze.com/> (único host público).

9. Criterios de suspensión (aceptación, devolución, suspensión)

Aceptación: El sitio se dará por aceptado cuando todos los scripts de prueba se ejecuten con éxito.

Devolución a desarrollo: para el reto no aplica

Suspensión de pruebas: En las siguientes condiciones:

- No hay acceso al sitio web
- - Datos incorrectos
 - Build defectuosa
 - Tasa alta de fallos críticos

10. Tipos de pruebas y alcance

- **Smoke:** entrada al sitio, abrir modales, navegar categorías, abrir un producto.
- **Funcionales/Flujo E2E:** sign up → login → agregar al carrito → checkout.
- **Negativas:** credenciales inválidas; checkout sin datos; agregar producto duplicado.

- **Regresión:** suite estable de los 10 escenarios (abajo), ejecutable en cada cambio.
- **Compatibilidad:** Chrome/Chromium y Firefox con WebdriverIO; Chromium/Firefox/WebKit con Playwright. [\[webdriver.io\]](https://webdriver.io), [\[playwright.dev\]](https://playwright.dev)
- **Accesibilidad (exploratoria):** foco, tabulación y etiquetas en modales.

11. Datos de prueba

- Usuario aleatorio por build para **Sign up** (evitar conflictos de duplicados).
- Productos: usar nombres visibles (p. ej., "Sony vaio i5", "Samsung galaxy s6").

12. Métricas y criterios de salida

- **Tasa de paso** de la suite $\geq 95\%$ en navegadores objetivo.
- **Defectos críticos** (checkout/login caídos) = 0 para aprobar release.
- **Tiempo total** de ejecución: < 10 min en CI con paralelismo.

13. Matriz de riesgos

Identifica riesgos potenciales, su impacto y cómo mitigarlos.

Riesgo	Probabilidad	Impacto	Severidad	Plan de mitigación
Cambios en selectores del DOM de modales (login/signup/contact)	Media	Alta	Alta	Centralizar selectores en Page Objects , preferir selectores por texto/rol/ids estables; agregar pruebas smoke que alerten rápido tras cambios y versionar los localizadores críticos.

Datos de usuario duplicados en Sign up	Alta	Media	Alta	
Cambios puntuales del contenido/catálogo	Baja	Media	Media	
Diferencias entre navegadores (Chrome vs Firefox)	Media	Media	Media	
Retrasos en la entrega de requisitos	Media	Alta	Alta	Alinear calendario y definir cut-off de requisitos
Inestabilidad del ambiente	Baja	Alto	Crítica	Monitoreo constante
Falta de datos de prueba	Baja	Alto	Media	Generación de datos
Cambios no controlados en código	Baja	Alta	Alta	Políticas de versionamiento y revisiones de código

* La severidad se calcula como combinación de probabilidad × impacto.

#	Riesgo	Prob.	Impacto	Severidad	Plan de mitigación
1	Cambios en selectores del DOM de modales (login/signup/contact)	Media	Alta	Alta	Centralizar selectores en Page Objects , preferir selectores por texto/rol/ids estables; agregar pruebas smoke que alerten rápido tras

					cambios y versionar los localizadores críticos.
2	Intermitencia del servicio Selenium local (drivers/puertos)	Media	Media	Media	Usar @wdio/selenium-standalone-service con versiones fijas y JDK presente; logs dedicados y reintentos de arranque; opción de fallback a ejecución headless en CI.
3	Diferencias entre navegadores (Chrome/Firefox/WebKit)	Media	Media	Media	Ejecutar en matriz de navegadores; aislar flakies por navegador; usar esperas explícitas y no asumir timing idéntico entre engines.
4	Datos de usuario duplicados en Sign up	Alta	Media	Alta	Generar usernames únicos por ejecución (timestamp/uuid) y validar el mensaje de duplicado; limpieza periódica del entorno si aplica.
5	Cambios de catálogo/contenido	Baja	Media	Media	Seleccionar productos por nombre visible con fallback por categoría/índice ; mantener pruebas de contrato a nivel UI para la rejilla.

#	Riesgo	Prob.	Impacto	Severidad	Plan de mitigación
6	Alertas del sitio bloquean el flujo (timing/auto-dismiss)	Media	Alta	Alta	Sincronizar con eventos de diálogo y timeouts prudentes; encapsular manejo de alertas en helpers reutilizables.

7	Inestabilidad por dependencias del SO al instalar navegadores Playwright	Media	Alta	Alta	Preinstalar browsers en imagen de CI (Containerfile) y cachear descarga; pin de versiones.
8	Flujos E2E largos sensibles a latencia de red	Media	Media	Media	Dividir en subflujos verificables, usar screenshots/trace/video en fallos y reintentos controlados en CI.
9	Allure CLI indisponible o falla por falta de Java	Baja	Alta	Media	Incluir OpenJDK 17 +Allure en imagen Podman; validación previa <code>allure --version</code> en stage de preparación.
10	Falsos positivos por timings (esperas insuficientes)	Media	Media	Media	Estandarizar waits con condiciones de estado (visible, enabled), no sleeps fijos; linters/PR checks.
11	Cambios en la estructura del modal “About us” (video/iframe)	Baja	Media	Baja	Verificar existencia del contenedor y fallback a chequeos de texto; no depender del player.
12	El pipeline no archiva artefactos grandes (traces/videos)	Baja	Media	Baja	Comprimir antes de archivar; retención configurable por tipo de job (smoke vs nightly).
13	Incompatibilidad puntual de driver/versión de navegador	Baja	Alta	Media	Versionar drivers, usar matriz y pin en services (Selenium/Playwright); smoke previo que valide compatibilidad.
14	Errores de permisos en contenedor (UID/GID/SELinux)	Media	Media	Media	Ejecutar contenedor con <code>-u \$(id -u):\$(id -g)</code> y <code>bind :z</code> si SELinux; documentar flags en README.

15	Regresiones por cambios de terceros (CDN, fonts, scripts)	Baja	Media	Baja	Minimizar aserciones visuales frágiles; validar solo funcionalidad y contenido mínimo necesario; smoke posterior al deploy.
----	--	------	-------	------	--

b. Riesgos positivos (Oportunidades)

#	Oportunidad (riesgo positivo)	Prob.	Impacto	Severidad	Estrategia para explotarlo al máximo
01	Ejecuciones paralelas en matriz de navegadores reducen TTE	Alta	Alta	Alta	Aumentar workers /shards y paralelizar por proyecto (Chromium/Firefox/WebKit); priorizar suites smoke y ejecutar full-regression nocturna.
02	Allure + JUnit brindan visibilidad y trazabilidad	Alta	Media	Alta	Automatizar Allure generate por CLI, almacenar reportes como artefactos y habilitar tendencias (history); consumir JUnit para calidad continua.
03	Contenedor Podman garantiza reproducibilidad	Media	Alta	Alta	Mantener Containerfile con Node 20 + Java 17 + browsers; “pin” de versiones; reusar imagen para todos los stages (build once, run many).
04	Page Objects reducen mantenimiento ante cambios de UI	Alta	Media	Media	Refactor por componentes, naming consistente, linters y revisiones de PR centradas en selectores ; mejorar reutilización.
05	Artefactos de Playwright (trace/video) aceleran diagnóstico**	Media	Media	Media	Activar <code>trace/video: 'retain-on-failure'</code> , políticas de retención por tipo de job y anexar enlaces directos en reportes Allure/CI para triage rápido.

14. Escenarios de prueba BDD (Gherkin)

A continuación, se encuentra la definición de los escenarios de prueba utilizando Gherkin:

1. Escenario: Registro de usuario exitoso

Dado que el usuario no está registrado

Y el usuario abre la página principal y presiona la opción "Sign up"

Cuando se registra con un usuario y contraseña válidos (demoQA / demo123)

Entonces debe visualizar un alert con el texto "Sign up successful"

Y debe volver a la página principal sin estar logueado, con la opción de "Log in" visible

2. Escenario: Registro con usuario ya existente

Dado que el usuario no está logueado

Cuando abre el modal de "Sign up"

Y se intenta registrar como un usuario existente (demoQA / loquesea)

Entonces se muestra una alert indicando "This user already exist" (SIC)

3. Escenario: Login exitoso

Dado que el usuario está registrado

Cuando ingresa credenciales válidas (demoQA / demo123)

Entonces debe volver a la misma página home pero con la opción de Logout y el mensaje de "Welcome demoQA"

4. Escenario: Navegación por categorías

Dado que el usuario está logueado como DemoQA

Cuando selecciona la categoría "Laptops"

Entonces se muestran productos relacionados: Sony vaion, MacBook air

5. Escenario: Agregar producto al carrito

Dado que el usuario está logueado como DemoQA

Cuando selecciona la categoría "Laptops"

Y selecciona el producto "Sony vaio i7"

Y lo agrega al carrito

Entonces se debe mostrar un alert que indique "Product Added"

Y al abrir el carrito el producto está en la lista

6. Escenario: Realizar compra

Dado que el usuario está logueado como DemoQA

Y el usuario tiene productos en el carrito

Cuando completa el formulario de compra

Entonces debe visualizar un mensaje de confirmación de compra exitosa indicando "Thank you for your purchase!"

7. Escenario: Logout

- Dado que el usuario está logueado como DemoQA

Cuando hace clic en "Log out"

Entonces debe regresar a la pantalla inicial sin sesión activa, con la opción de "Log in" en la parte superior

8. Escenario: Inicio de sesión inválido

Dado que el usuario no está logueado

Y abre la funcionalidad de "Log in"

Y intento iniciar sesión con credenciales inválidas (demoQA / loquesea)

Entonces veo una alerta que indica "Wrong password."

9. Escenario: Filtrado por categoría (Phones)

Cuando navego a la categoría "Phones"

Entonces solo veo productos de la categoría "Phones"

10. Escenario: Agregar un producto al carrito desde el detalle

Dado que navego a la categoría "Laptops"

Cuando abro el detalle del producto "Sony vaio i5"

Y agrego el producto al carrito

Entonces veo una alerta de agregado exitoso

11. Escenario: Eliminar un producto del carrito

Dado que tengo un producto en el carrito

Cuando elimino el producto del carrito

Entonces el carrito queda vacío

12. Escenario: Completar la compra (Place Order)

Dado que tengo productos en el carrito

Cuando presiono "Place Order" y completo mis datos

Y confirmo la compra

Entonces veo el mensaje de confirmación de compra

13. Escenario: Enviar el formulario de Contacto

Cuando abro el modal "Contact"

Y completo el formulario y envío

Entonces recibo una confirmación/alerta de envío

14. Escenario: Ver el modal "About us"

Cuando abro el modal "About us"

Entonces visualizo el contenido del modal (incluye video)